OpenDeal

Bitcoin-native commerce coordination protocol

Hackathon concept and implementation proposal | DevFest × Btrust | Team review draft

Status: Proposed MVP — not yet implemented. This document outlines an achievable four-day build, technical choices, security boundaries, and decisions for the team.

1. Executive summary

OpenDeal is an open-source protocol for coordinating peer-to-peer commerce using Bitcoin. Buyers and sellers create signed, verifiable agreements; a Bitcoin multisig escrow secures payment; and a pre-agreed mediator can assist when a dispute occurs. The API coordinates deals but must never control the private keys required to spend escrow funds.

Our differentiation is not escrow alone. It is portable deal receipts, a reusable Rust coordination library, clearly defined deal states, and a Bitcoin settlement adapter that other applications can integrate.

2. Problem and target users

Peer-to-peer trades arranged through messaging apps often require one party to take the first risk: the buyer pays before delivery or the seller delivers before payment. Centralized escrow adds another trusted party and may introduce custody, access, or dispute concerns.

Initial use case: a buyer purchases a used laptop from a seller found through WhatsApp. The parties agree on price, delivery deadline, and a mediator, then use OpenDeal to coordinate payment and evidence of delivery.

3. Proposed user journey

Stage	Expected action / proof
Create	Seller proposes item, price in sats, deadline, mediator, and delivery terms.
Accept	Buyer and seller sign the same canonical agreement; derive a verifiable deal ID.
Fund	Buyer funds a 2-of-3 multisig output; the system verifies amount, output and confirmations.
Deliver	Seller records delivery evidence; buyer accepts or disputes.
Settle	Buyer + seller sign a PSBT to pay the seller; or mediator + one party sign a refund/payout.
Verify	Downloadable receipt records signed agreement, signed events, and Bitcoin transaction IDs.

4. Core MVP features

- Rust protocol crate: deal schema, authorized state transitions, deterministic serialization, agreement hashing, signatures, and event verification.
- Bitcoin adapter: native SegWit P2WSH 2-of-3 multisig, regtest funding verification, PSBT creation, signing coordination, and broadcast.
- Axum API: deal creation/acceptance, signed event submission, funding status, settlement proposal and dispute records.
- PostgreSQL persistence: deals, parties, mediator, event chain, evidence references, and settlement metadata.
- Next.js demonstration: buyer, seller and mediator views; timeline, escrow status, settlement and receipt export.

5. Architecture and repository layout

Frontend (Next.js) → Axum API → opendeal-core (Rust protocol) → opendeal-bitcoin (Bitcoin Core RPC / PSBT). PostgreSQL stores application data and event records. Participant keys stay outside the API and database.

Proposed workspace: crates/opendeal-core (deal.rs, state.rs, agreement.rs, event.rs, verification.rs); crates/opendeal-bitcoin (escrow.rs, psbt.rs, settlement.rs, rpc.rs); crates/opendeal-api (routes, handlers, services, db); web/; examples/regtest-demo/; docs/protocol.md and docs/threat-model.md.

6. Technical and security decisions

- Use three independently controlled Bitcoin Core regtest wallets for buyer, seller and mediator. No seed phrases or private keys in the server.
- Use a 2-of-3 P2WSH multisig output and PSBTs for collaborative spending. Validate amounts, fees, addresses, input UTXOs, and outputs before signatures.
- Canonicalize the agreement before hashing; both parties sign the exact same bytes. Each event includes the prior event hash and an authorized signature.
- Funding is not final just because a transaction is seen: track confirmations and ensure the escrow output matches the agreed script and value.
- A mediator cannot spend funds alone, but mediator collusion and non-cooperation remain risks. A mediator decision does not itself move Bitcoin.
- Timeout and unilateral recovery paths are not part of the four-day core unless thoroughly designed and tested. Clearly label the MVP experimental and regtest-only.

7. Four-day delivery plan

Day	Focus	Exit criterion
1	Rust workspace, deal state machine, signed agreement/event format, DB and API	Two parties can create, sign and independently verify a deal.
2	Regtest wallets, multisig descriptor,	Working on-chain funding and

	funding, PSBT, two signatures and broadcast	successful settlement demo.
3	Frontend, deal timeline, receipt export, disputes and mediator-assisted refund	End-to-end successful and disputed trade scenarios.
4	Security tests, integration fixes, docs, pitch and backup recording	Reproducible demo and understandable open-source repository.

8. Scope management

Must ship	Strong bonus	Defer
Create/accept deal; verify signatures	Mediator-assisted refund	Lightning adapter
Bitcoin funding and payout	Portable receipt verifier CLI	Multiple payment networks
Minimal web demo and README	Structured evidence uploads	Automated timelock recovery
Tests for invalid transitions	More polished dashboard	Production deployment and real funds

9. Demonstration and judging narrative

Suggested four-minute demo: 0:00–0:40 trust problem; 0:40–1:20 create/sign deal; 1:20–2:10 fund escrow; 2:10–2:50 jointly sign and broadcast payout; 2:50–3:30 mediator-assisted refund; 3:30–4:00 export and independently verify receipt, show reusable Rust crate.

Judging message: OpenDeal is not another centralized marketplace. It is a reusable coordination layer that lets applications attach verifiable agreements and dispute workflows to real Bitcoin settlement.

10. Risks and mitigations

Risk	Mitigation
PSBT or script implementation takes too long	Prove funding + one successful payout by end of Day 2; use established libraries and Bitcoin Core RPC.
Complexity prevents a complete demo	Freeze MVP scope; keep the UI minimal; prioritize the happy path.
Mediator collusion / frozen funds	State limitations explicitly; use regtest only; document recovery design as future work.
AI-generated security bugs	Review all Bitcoin signing logic; add negative tests; inspect decoded transactions and PSBTs.
Demo infrastructure failure	Prepare seeded regtest scenario and backup screen recording.

11. Team review: decisions needed

- Confirm team size and owners for protocol/backend, Bitcoin integration, frontend, and testing/pitch.
- Agree on P2WSH 2-of-3 multisig for the demo rather than implementing Taproot script paths.
- Choose signed agreement identity format and receipt schema early on Day 1.
- Decide whether dispute/refund is required for the demo or a bonus after successful payout.
- Confirm hackathon rules, judging rubric, network requirements and whether regtest demos are accepted.

12. Proposed submission description

OpenDeal is an open-source Bitcoin transaction coordination protocol for peer-to-peer commerce. It enables buyers and sellers to create cryptographically verifiable agreements, secure payments with collaborative Bitcoin multisig escrow, and coordinate structured dispute resolution without giving a centralized intermediary unilateral control of funds. Its Rust library and API are designed for reuse by marketplaces and other commerce applications.

Review note: This is a design proposal, not a security audit or a claim of production readiness. The project should not handle real user funds during the hackathon.